

6-1 Do-while statement

The **do-while loop in C++** executes a block of code at least once, and then repeatedly as long as a specified condition remains true. The key difference from a standard while loop is that the condition is checked *after* the loop body has run (an exit-controlled loop), not before.

Syntax

The syntax for a **do-while** loop is as follows:

```
do {  
    // code block to be executed  
}  
while (condition); // Note the required semicolon here
```

Example (1):

write program using **do-while loop** where a user is prompted to enter a positive number. The program will continue to prompt the user until they enter a valid number.

Sol.

```
#include <iostream>  
using namespace std;  
int main() {  
  
    int n;  
    // Do-while loop to ensure user enters a positive number  
  
    do {  
        cout << "Enter a positive number: ";  
        cin >> n;  
    } while (n <= 0);  
  
    cout << "Entered number: " << n << endl;  
  
    return 0;  
}
```

Output:

```
Enter a positive number: -4  
Enter a positive number: -105  
Enter a positive number: -89  
Enter a positive number: 4  
Entered number: 4
```

Example (2):

Using do-while, write C++ program to find the factorial of n:

$$n! = n * n-1 * n-2 * n-3 * \dots * 2 * 1$$

$$5! = 5 * 4 * 3 * 2 * 1$$

Sol.

```
#include <iostream>
using namespace std;

int main() {
    int number, fact = 1;
    cout << "enter positive number:";
    cin >> number;
    do
    {
        fact = fact * number;
        number --;
    }
    while ( number > 1 );
    cout << "factorial is: "<< fact;

    return 0;
}
```

Output:

enter positive number:5

factorial is: 120

Homework:

Using do-while, write C++ program to valid input checking, that accept the numbers between 50 ... 70 only.

6-2 Nested loops

A **nested loop** in C++ is a loop placed inside the body of another loop. This structure allows for repetitive tasks within each iteration of a larger process, such as processing multi-dimensional data structures like arrays or matrices, or printing patterns.

Syntax for Nested For loop:

```
for ( initialization; condition; increment ) {  
    for ( initialization; condition; increment ) {  
        // statement of inside loop  
    }  
    // statement of outer loop  
}
```

Syntax for Nested While loop:

```
while(condition) {  
    while(condition) {  
        // statement of inside loop  
    }  
    // statement of outer loop  
}
```

Syntax for Nested Do-While loop:

```
do{  
    do{  
        // statement of inside loop  
    }while(condition);  
    // statement of outer loop  
}while(condition);
```

Example (1):

Write a program that uses nested for loops to print a simple multiplication table (1 to 3):

```
#include <iostream>
using namespace std;
```

```
int main() {
    for (int i = 1; i <= 3; i++)
    {
        for (int j = 1; j <= 4; j++) {
            cout << i<<"*"<<j<<"="<<i * j << "\n";
        }
        cout << "\n";
    }

    return 0;
}
```

output:

```
1*1=1
1*2=2
1*3=3
1*4=4

2*1=2
2*2=4
2*3=6
2*4=8

3*1=3
3*2=6
3*3=9
3*4=12
```

Example (2):

Using a nested for loop, Write C++ program to evaluate the following series:

$$\sum_{i=1}^5 \sum_{j=1}^{10} i + 2j$$

Sol.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int i, j, sum= 0;
```

output
sum is:700

```
for ( i = 1; i <= 5; i ++ )
{
    for ( j = 1; j <= 10; j ++ )
    {
        sum= sum+ ( i + 2 * j );
    }
}
cout << "sum is:"<< sum;

return 0;
}
```

Try to change the position of "cout ".
What will you see.

Example (3)

Using nested while loop, write program to print the following pattern.

```
*
**
***
****
*****
*****
```

Sol.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int row = 1;
    int column = 1;

    while (row <= 6)
    {
        column = 1;
        while (column <= row)
        {
            cout << "*" ;
```

output

```
*
**
***
****
*****
*****
```

```
        column++;  
    }  
    cout << endl;  
    row++;  
}  
return 0;  
}
```

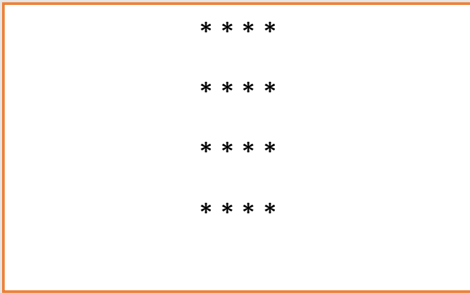
Example (4):

Write a program to print the following square pattern using nested **do-while** loops.

```
* * * *  
* * * *  
* * * *  
* * * *
```

Sol.

```
#include <iostream>  
using namespace std;  
int main() {  
    int i = 0;  
    // Outer loop to print each row  
    do {  
        int j = 0;  
  
        // Inner loop to print each character in each row  
        do {  
            cout << "* ";  
            j++;  
        } while (j < 4);  
        cout << endl;  
        i++;  
    } while (i < 4);  
  
    return 0;  
}
```



```
* * * *  
* * * *  
* * * *  
* * * *
```

6-3 Breaking control statements

Break

You have already seen the **break** statement used in an earlier lecture. It was used to "jump out" of a **switch** statement. The **break** statement can also be used to jump out of a **loop**.

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 0; i < 10; i++) {
        if (i == 4) {
            break;
        }
        cout << i << "\n";
    }
    return 0;
}
```

Output

0
1
2
3

Continue

The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop. This example skips the value of 4:

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 0; i < 10; i++) {
        if (i == 4) {
            continue;
        }
        cout << i << "\n";
    }
    return 0;
}
```

Output

0
1
2
3
5
6
7
8
9



Go to

The goto is a jump statement that transfers the control flow of a program to a labeled statement within the scope of the same function, breaking the normal flow of a program.

The following program illustrates how the goto statement can be used for error handling to avoid unnecessary calculations:

```
#include <iostream>
using namespace std;

int main() {
    double numerator = 10.0;
    double denominator = 0.0;
    double result;

    if (denominator == 0) {
        // Jump to cleanup if denominator is 0
        goto cleanup;
    }

    // Perform the division
    result = numerator / denominator;
    cout << "Result: " << result << endl;

cleanup:
    cout << "Error: Division by zero is not allowed." << endl;
    cout << "Program has finished executing." << endl;

    return 0;
}
```

Output
Error: Division by zero is not
allowed.
Program has finished executing.

Homework:

Using for, while, and do-while to *find the result of the following*:

1- $\sum_{i=1}^{20} a_i^2$

2- Print the following series: 1, 2, 4, 16, 32, 64